📘 SORTING STRATEGIES — THEORY + ALGORITHM + FLOW + EXAMPLES

🔹 What is Sorting? (Theory)

Sorting is the process of **arranging elements in a specific order** (ascending or descending) based on a comparison key.

Why sorting matters:

- Faster searching (binary search)
- Efficient data processing
- Foundation for many algorithms

🔹 Classification (Theory)

1️⃣ Comparison-based sorting

Uses element-to-element comparisons.
Lower bound: **Ω(n log n)**

2️⃣ Non-comparison sorting

Uses data properties (digits, range).
Can beat n log n.

1️⃣ Bubble Sort

📘 Theory

Bubble Sort repeatedly compares **adjacent elements** and swaps them if they are in the wrong order.
After each pass, the **largest element moves to the end** (like a bubble rising).

🧩 Algorithm

1. Traverse array
2. Compare adjacent elements

3. Swap if left > right
4. Repeat for remaining unsorted part

🔁 Flowchart

Start

|

|-- for i = 0 to n-1

| |

| |-- for j = 0 to n-i-2

| |

| |-- if arr[j] > arr[j+1]

| swap

|

End

🧪 Example

Array: [5, 1, 4, 2]

Pass 1: [1, 5, 4, 2]

Pass 2: [1, 4, 5, 2]

Pass 3: [1, 4, 2, 5]

Pass 4: [1, 2, 4, 5]

🧪 Example

[5, 1, 4, 2]

→ [1, 5, 4, 2]

→ [1, 4, 5, 2]

→ [1, 4, 2, 5]

→ [1, 2, 4, 5]

💻 Java Code

```
void bubbleSort(int[] arr) {
  for (int i = 0; i < arr.length; i++) {
    for (int j = 0; j < arr.length - i - 1; j++) {
      if (arr[j] > arr[j + 1]) {
        int temp = arr[j];
        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
      }
    }
  }
}
```

⏱️ Complexity

- Time: $O(n^2)$
- Space: O(1)
- Stable: ✅

🎯 Use Case

❌ Not used in real systems
✅ Educational purposes only

2️⃣ Selection Sort

📘 Theory

Selection Sort repeatedly **selects the smallest element** from the unsorted part and places it at the correct position.

Key idea: **minimum selection + swap**

🧩 Algorithm

1. Find minimum element
2. Swap with first unsorted index
3. Move boundary forward

🔁 Flowchart

Start

|

|-- for i = 0 to n-1

| |

| |-- find min from i to n-1

| |

| |-- swap min with arr[i]

|

End

🧪 Example

[64, 25, 12, 22]

Step 1: [12, 25, 64, 22]

Step 2: [12, 22, 64, 25]

Step 3: [12, 22, 25, 64]

🧪 **Example**

[64, 25, 12, 22]

→ [12, 25, 64, 22]

→ [12, 22, 64, 25]

→ [12, 22, 25, 64]

💻 **Java Code**

```
void selectionSort(int[] arr) {
  for (int i = 0; i < arr.length; i++) {
    int min = i;
    for (int j = i + 1; j < arr.length; j++) {
      if (arr[j] < arr[min]) min = j;
    }
    int temp = arr[min];
    arr[min] = arr[i];
    arr[i] = temp;
  }
}
```

⏱️ **Complexity**

- Time: $O(n^2)$
- Space: O(1)
- Stable: ❌

🎯 Use Case

- When **memory writes are costly**
- Embedded systems

3️⃣ Insertion Sort

📘 Theory

Insertion Sort builds a sorted array **one element at a time** by inserting each element into its correct position.

Similar to sorting playing cards.

🧩 Algorithm

1. Pick next element
2. Shift larger elements right
3. Insert element in correct position

🔄 Flowchart

Start

|

|-- for i = 1 to n-1

| |

| |-- key = arr[i]

| |-- shift elements > key

| |-- insert key

|

End

🧪 **Example**

[5, 3, 4]

Insert 3 → [3, 5, 4]

Insert 4 → [3, 4, 5]

🧪 **Example**

[5, 3, 4]

→ [3, 5, 4]

→ [3, 4, 5]

💻 **Java Code**

```
void insertionSort(int[] arr) {
   for (int i = 1; i < arr.length; i++) {
      int key = arr[i];
      int j = i - 1;
      while (j >= 0 && arr[j] > key) {
         arr[j + 1] = arr[j];
         j--;
      }
      arr[j + 1] = key;
   }
}
```

⏱️ **Complexity**

- Best: O(n)

- Worst: $O(n^2)$
- Space: O(1)
- Stable: ✅

🎯 Use Case

- Nearly sorted arrays
- Small datasets

🔽 Merge Sort

📘 Theory

Merge Sort follows **Divide and Conquer**:

- Divide array into halves
- Sort each half
- Merge sorted halves

Sorting happens during **merge phase**.

🧩 Algorithm

1. Divide array until single elements
2. Merge sorted subarrays
3. Repeat until full array is sorted

🔁 Flowchart

Start

|

|-- divide array

| |

| |-- sort left

| |-- sort right

| |

|-- merge sorted halves

|

End

🧪 **Example**

[5,2,4,1]

→ [5,2] [4,1]

→ [2,5] [1,4]

→ [1,2,4,5]

💻 **Java Code (Core)**

```
void mergeSort(int[] arr, int l, int r) {
   if (l < r) {
      int m = (l + r) / 2;
      mergeSort(arr, l, m);
      mergeSort(arr, m + 1, r);
      merge(arr, l, m, r);
   }
}
```

🧪 **Example**

[5, 2, 4, 1]

→ [5,2] [4,1]

→ [2,5] [1,4]

→ [1,2,4,5]

⏱️ Complexity

- Time: O(n log n)
- Space: O(n)
- Stable: ✅

🎯 Use Case

- Linked lists
- External sorting
- Guaranteed performance systems

5️⃣ Quick Sort

📘 Theory

Quick Sort selects a **pivot**, partitions array, and recursively sorts subarrays.

Sorting happens during **partitioning**.

🧩 Algorithm

1. Choose pivot
2. Partition elements around pivot
3. Recursively sort partitions

🔁 Flowchart

Start

|

|-- choose pivot

|-- partition array

| |

| |-- left < pivot

| |-- right > pivot

|

|-- recursive sort

|

End

🧪 **Example**

[5,2,4,1]

Pivot = 2

→ [1] 2 [5,4]

→ [1,2,4,5]

💻 **Java Code (Core)**

```
int partition(int[] arr, int low, int high) {
   int pivot = arr[high];
   int i = low - 1;
   for (int j = low; j < high; j++) {
      if (arr[j] < pivot) {
         i++;
         int t = arr[i]; arr[i] = arr[j]; arr[j] = t;
      }
```

```
    }
    int t = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = t;
    return i + 1;
}
```

🧪 **Example**

[5, 2, 4, 1]

Pivot = 2

→ [1] 2 [5,4]

→ [1,2,4,5]

⏱️ **Complexity**

- Average: O(n log n)
- Worst: $O(n^2)$
- Space: O(log n)
- Stable: ❌

🎯 **Use Case**

- General-purpose sorting
- In-memory arrays

6️⃣ **Heap Sort**

📘 **Theory**

Heap Sort uses a **binary heap**:

- Build max heap

- Repeatedly extract max element

🧩 Algorithm

1. Build max heap
2. Swap root with last
3. Heapify reduced heap

🔁 Flowchart

Start

|

|-- build heap

|

|-- extract max

|-- heapify

|

End

💻 Java Code (Core)

```
void heapSort(int[] arr) {
   PriorityQueue<Integer> pq = new PriorityQueue<>();
   for (int n : arr) pq.add(n);
   for (int i = 0; i < arr.length; i++) arr[i] = pq.poll();
}
```

⏱️ Complexity

- Time: O(n log n)

- Space: O(1)
- Stable: ❌

🎯 Use Case

- Worst-case guaranteed sorting
- Priority-based systems

7️⃣ Counting Sort (Non-Comparison)

📘 Theory

Counts frequency of elements instead of comparing.

🧩 Algorithm

1. Count frequency
2. Rebuild sorted array

🧪 Example

[1,4,1,2]

Count: [0,2,1,0,1]

Sorted: [1,1,2,4]

💻 Java Code

```
void countingSort(int[] arr, int max) {
   int[] count = new int[max + 1];
   for (int n : arr) count[n]++;
   int idx = 0;
   for (int i = 0; i <= max; i++)
      while (count[i]-- > 0)
         arr[idx++] = i;
}
```

⏱️ Complexity

- Time: O(n + k)
- Space: O(k)
- Stable: ✅

🎯 Use Case

- Small integer ranges

🔚 FINAL SUMMARY TABLE

Algorithm	Time	Space	Stable	Best Use
Bubble	$O(n^2)$	O(1)	✅	Learning
Selection	$O(n^2)$	O(1)	❌	Minimal swaps
Insertion	$O(n^2)$	O(1)	✅	Nearly sorted
Merge	O(n log n)	O(n)	✅	Linked lists

Algorithm	Time	Space	Stable	Best Use
Quick	O(n log n)	O(log n)	❌	General
Heap	O(n log n)	O(1)	❌	Worst-case
Counting	O(n+k)	O(k)	✅	Integers

🎯 ONE-LINE INTERVIEW SUMMARY

“Sorting algorithms differ in comparison strategy, memory usage, stability, and performance guarantees. The choice depends on data size, structure, and constraints.”